

COURSE CODE

SBT-DF203

COURSE TITLE

Basic Networking Skills for Digital Forensics

LAB TITLE

DNS Spoofing Forensics

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

24/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

Mr. Aminu Idris, AMCPN
Commandant

Executive Summary

This report describes the DNS Spoofing Forensics exercise (SBT-DF203 Lab 8) conducted by Kumpon Emmanuel in a controlled virtual lab environment. The exercise used approved tools and a special training domain (portal.icdfa.test). It was done in accordance with the ICDFA lab's rules on legality, ethics, and safety. The activity was authorized only for this lab; no actual production, campus, office, or public network traffic was involved.

As of this draft, I have finished setting up the environment, collecting baseline evidence, and reviewing the pre-flight script. However, I have not yet completed the controlled DNS-spoofing capture or the forensic analysis (Parts C-E of the lab manual).

Lab Network Diagram

Role	Hostname / VM	IP Address	MAC / Notes
Analyst VM	ponzhi	192.168.109.128/24	eth0; hosts training page via Apache2
Gateway	lab virtual gateway	192.168.109.2	lladdr 00:50:56:fe:1f:71 (from arp_before.txt)
Additional neighbor	unidentified	192.168.109.254	lladdr 00:50:56:ff:df:3b STALE identify before proceeding
Victim VM	Pending	not yet confirmed	Must be obtained via IP-Br address on the victim VM itself

Environment Setup

The lab evidence folder structure was created under ~/SBT-DF203-Lab8, and required packages were installed (apache2, tshark, wireshark, python3-scapy, python3-pip, build-essential, python3-dev, libnetfilter-queue-dev).

```
(ponzhi@kali) - [~/SBT-DF203-Lab8]
$ sudo apt update
[sudo] password for ponzhi:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [54.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [270 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Fetched 77.2 MB in 55s (1,402 kB/s)
470 packages can be upgraded. Run 'apt list --upgradable' to see them.

(ponzhi@kali) - [~/SBT-DF203-Lab8]
$
```

```
(ponzhi@kali) - [~/SBT-DF203-Lab8]
$ sudo apt install -y apache2 tshark wireshark python3-scapy python3-pip \
  build-essential python3-dev libnetfilter-queue-dev
tshark is already the newest version (4.6.6-1).
wireshark is already the newest version (4.6.6-1).
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-pip is already the newest version (26.1.2+dfsg-1).
python3-pip set to manually installed.
build-essential is already the newest version (12.12).
build-essential set to manually installed.
The following packages were automatically installed and are no longer required:
  bluez-hcidump      libbluray3      libopenjp0.27   libvidstab1.1   python3.13-dev
  firmware-marvell-presteria libhdf5-310     libperl5.40     libx265-215     python3.13-minimal
  iucode-tool        libhdf5-hl-310  libpoppler156   perl-modules-5.40 python3.13-tk
  libabsl20260107    libntfs-3g89t64 libpython3.13-dev python3.13       python3.13-venv
Use 'sudo apt autoremove' to remove them.

Upgrading:
  apache2      libpython3-dev      libpython3.14-minimal  python3-minimal  python3.14-dev
  apache2-bin  libpython3-stdlib  libpython3.14-stdlib  python3-tk       python3.14-minimal
  apache2-data libpython3.14      python3               python3-venv     python3.14-tk
  apache2-utils libpython3.14-dev  python3-dev           python3.14      python3.14-venv

Installing:
  libnetfilter-queue-dev
```

```
ponchi@kali: ~/SBT-DF203-Lab8
Session Actions Edit View Help
Setting up python3 (3.14.7-3)...
running python rtupdate hooks for python3.14...
/usr/share/commix/src/core/tamper/backslashes.py:21: SyntaxWarning: "\"" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\\""? A raw string is also an option.
  About: Adds back slashes (\) between the characters in a given payload.
/usr/share/fern-wifi-cracker/core/functions.py:100: SyntaxWarning: 'return' in a 'finally' block
  return read_file
running python post-rtupdate hooks for python3.14...
Setting up pkgconf:amd64 (2.5.1-4)...
Setting up python3.14-venv (3.14.7-3)...
Setting up python3-venv (3.14.7-3)...
Setting up python3-dev (3.14.7-3)...
Setting up libnfnetwork-dev:amd64 (1.0.2-3+b2)...
Setting up libnetfilter-queue-dev:amd64 (1.0.5-4+b2)...
Processing triggers for kali-menu (2026.3.2)...
Processing triggers for desktop-file-utils (0.28-1)...
Processing triggers for doc-base (0.11.2)...
Processing 1 changed doc-base file...
Processing triggers for libc-bin (2.42-17)...
Processing triggers for systemd (261.2-1)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for mailcap (3.77)...

(ponchi@kali)~[~/SBT-DF203-Lab8]
$
```

Training Page, Interfaces, Routes, and ARP Baseline

The harmless ICDDFA training warning page was set up in the Apache2 web root on the analyst VM and secured with a hash. Before starting any spoofing, we recorded the interface, routing, and ARP neighbor states.

```
(ponchi@kali)~[~/SBT-DF203-Lab8]
$ sudo systemctl enable --now apache2

(ponchi@kali)~[~/SBT-DF203-Lab8]
$ printf '<!DOCTYPE html>\n<html><body style="font-family:Arial"><h1>ICDDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Kumpon Emmanuel</p></body></html>\n' \
| sudo tee /var/www/html/index.html
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Kumpon Emmanuel</p></body></html>

(ponchi@kali)~[~/SBT-DF203-Lab8]
$ sha256sum /var/www/html/index.html | tee reports/training_page_sha256.txt
6c3b8d88e651a84c7ec7100239506a70946343350e189f3c627ff3d5be95fc54 /var/www/html/index.html

(ponchi@kali)~[~/SBT-DF203-Lab8]
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.109.128/24 fe80::20c:29ff:fe6f:fd97/64

(ponchi@kali)~[~/SBT-DF203-Lab8]
$ ip route | tee reports/routes.txt
default via 192.168.109.2 dev eth0 proto dhcp src 192.168.109.128 metric 100
192.168.109.0/24 dev eth0 proto kernel scope link src 192.168.109.128 metric 100

(ponchi@kali)~[~/SBT-DF203-Lab8]
$ ip neigh show | tee reports/arp_before.txt
192.168.109.254 dev eth0 lladdr 00:50:56:ff:df:3b STALE
192.168.109.2 dev eth0 lladdr 00:50:56:fe:1f:71 STALE
```

Training page SHA-256	6c3b8d88e651a84c7ec7100239506a70946343350e189f3c627ff3d5be95fc54
Analyst interface / IP	eth0 192.168.109.128/24
Default gateway	192.168.109.2
ARP baseline (arp_before.txt)	192.168.109.254 -> 00:50:56:ff:df:3b (STALE); 192.168.109.2 -> 00:50:56:fe:1f:71 (STALE)

Part A — Baseline DNS/ARP/HTTP Capture

A 30-second network capture was taken on eth0 and saved as evidence/dns_baseline.pcapng. This capture was filtered to show only the analyst's own traffic patterns. Once the background tshark process finished, the file was hashed to maintain a chain of custody, which was confirmed by waiting before hashing.

```

(ponzhi@kali)~[/SBT-DF203-Lab8]
$ tshark -i eth0 -f 'host 192.168.109.128 and (arp or port 53 or tcp port 80)' \
-a duration:30 -w evidence/dns_baseline.pcapng &
sleep 3
[2] 40908
Capturing on 'eth0'

(ponzhi@kali)~[/SBT-DF203-Lab8]
4 tswait
sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt

[2] done      tshark -i eth0 -f 'host 192.168.109.128 and (arp or port 53 or tcp port 80)'
(ponzhi@kali)~[/SBT-DF203-Lab8]
$ wait
sha256sum evidence/dns_baseline.pcapng | tee reports/dns_baseline_sha256.txt
d899c96aeb4d5d154075a40aa606e2802880f4ed5c865516dbad400ee52c736b  evidence/dns_baseline.pcapng

(ponzhi@kali)~[/SBT-DF203-Lab8]
$

```

During the capture window, we issued a dig query for the training domain. The query returned NXDOMAIN, which means there was no response because we had not set up a hosts-file entry or an authoritative zone for portal.icdfa.test at that time. This is a gap in our baseline evidence, not a sign of spoofing.

```
(ponzhi@kali)-[~]
$ dig portal.icdfa.test A

; <<>> DiG 9.20.27-2-Debian <<>> portal.icdfa.test A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NXDOMAIN, id: 51493
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;portal.icdfa.test.      IN      A
;
;; AUTHORITY SECTION:
;      5      IN      SOA      a.root-servers.net. nstld.verisign-grs.com. 2026092400 1800 900 604800 86400

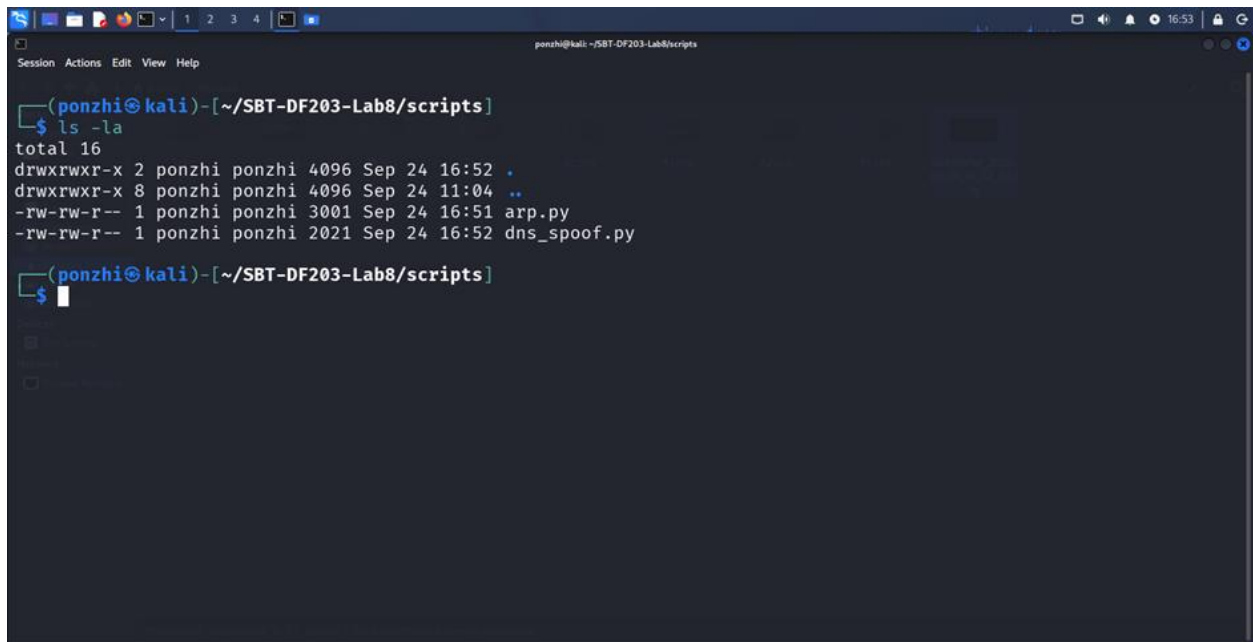
;; Query time: 4012 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Thu Sep 24 11:47:38 WAT 2026
;; MSG SIZE rcvd: 121

(ponzhi@kali)-[~]
$
```

Baseline capture file	evidence/dns_baseline.pcapng
Baseline capture SHA-256	d899c96aeb4d5d154075a40aa606e2802880f4ed5c865516dbad400ee52c736b
Baseline DNS result	NXDOMAIN
Legitimate responder IP / MAC / TTL / response time	

Sourcing and Placing the Instructor-Approved Scripts

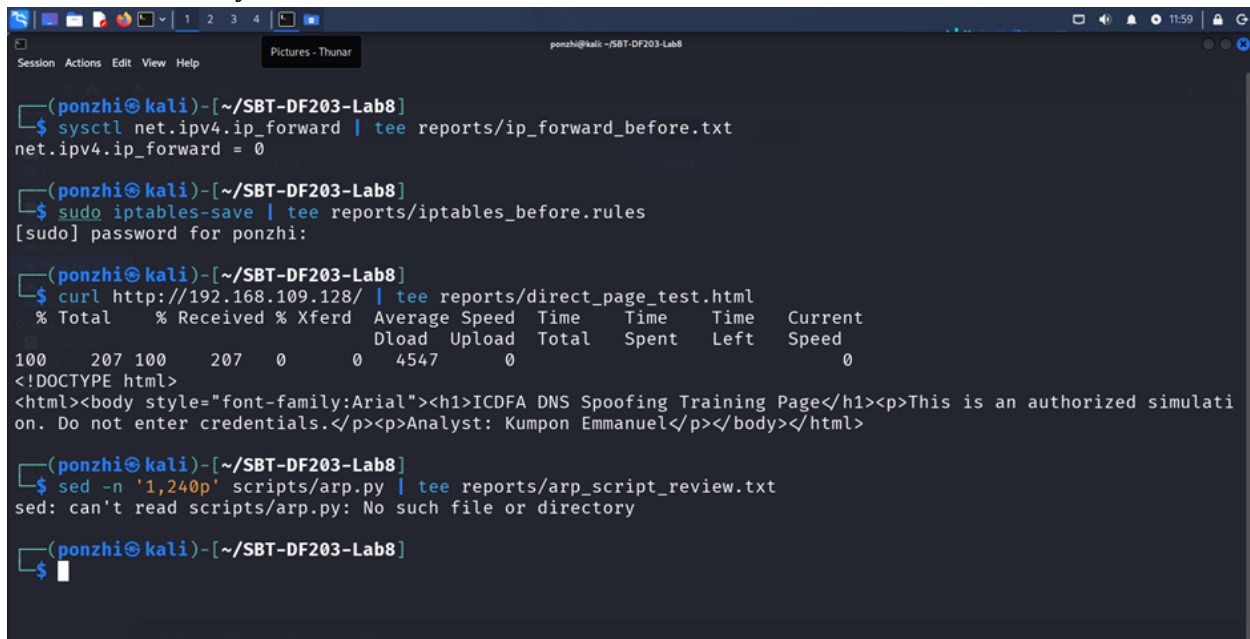
arp.py and dns_spoof.py were retrieved from a public academic digital-forensics course repository (frankwxu/digital-forensics-lab on GitHub) and placed into ~/SBT-DF203-Lab8/scripts/.

A terminal window with a dark background and light blue text. The window title is 'ponzhi@kali: ~/SBT-DF203-Lab8/scripts'. The prompt is '(ponzhi@kali)-[~/SBT-DF203-Lab8/scripts]'. The user has entered the command 'ls -la'. The output shows the directory contents: 'total 16', 'drwxrwxr-x 2 ponzhi ponzhi 4096 Sep 24 16:52 .', 'drwxrwxr-x 8 ponzhi ponzhi 4096 Sep 24 11:04 ..', '-rw-rw-r-- 1 ponzhi ponzhi 3001 Sep 24 16:51 arp.py', and '-rw-rw-r-- 1 ponzhi ponzhi 2021 Sep 24 16:52 dns_spoof.py'. The prompt is now '\$ ' with a cursor.

```
(ponzhi@kali)-[~/SBT-DF203-Lab8/scripts]
$ ls -la
total 16
drwxrwxr-x 2 ponzhi ponzhi 4096 Sep 24 16:52 .
drwxrwxr-x 8 ponzhi ponzhi 4096 Sep 24 11:04 ..
-rw-rw-r-- 1 ponzhi ponzhi 3001 Sep 24 16:51 arp.py
-rw-rw-r-- 1 ponzhi ponzhi 2021 Sep 24 16:52 dns_spoof.py
$
```

Part B

Before running any scripts, we recorded the settings for IP forwarding and the firewall (iptables). We also confirmed that we could directly access the training page using its IP address by using curl from the analyst's side.

A terminal window titled 'ponzhi@kali: ~/SBT-DF203-Lab8' showing a series of commands and their outputs. The commands include checking and setting IP forwarding, saving iptables rules, and using curl to fetch a web page. The curl output shows a successful download of an HTML page with a warning about the document type.

```
(ponzhi@kali)-[~/SBT-DF203-Lab8]
$ sysctl net.ipv4.ip_forward | tee reports/ip_forward_before.txt
net.ipv4.ip_forward = 0

(ponzhi@kali)-[~/SBT-DF203-Lab8]
$ sudo iptables-save | tee reports/iptables_before.rules
[sudo] password for ponzhi:

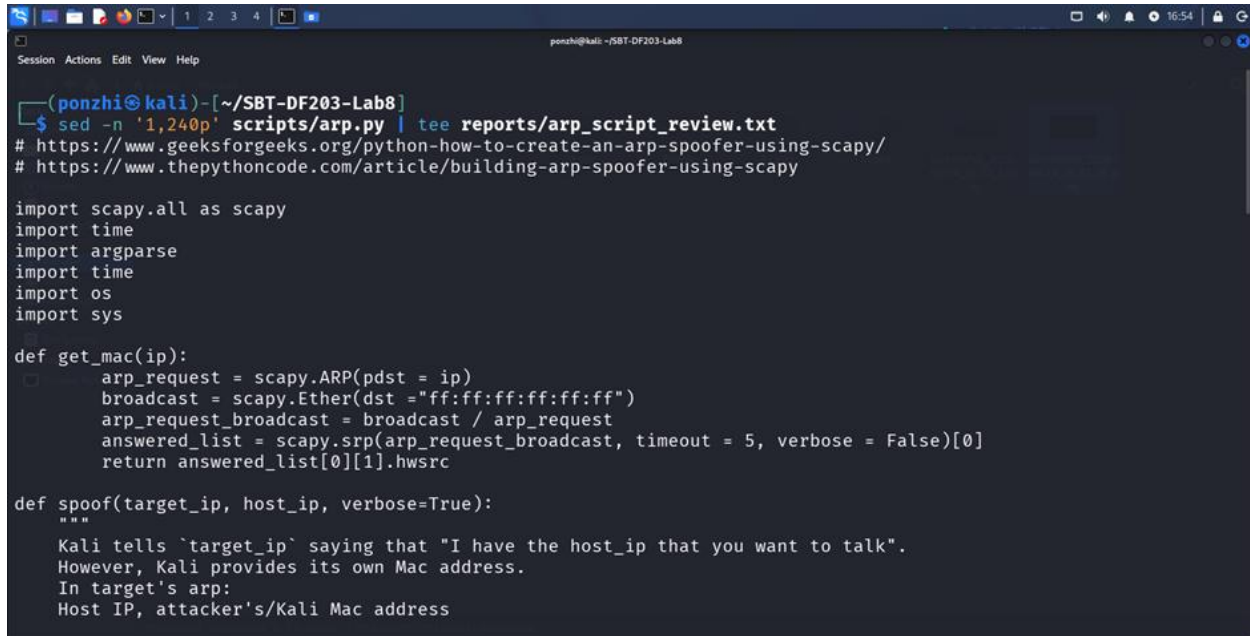
(ponzhi@kali)-[~/SBT-DF203-Lab8]
$ curl http://192.168.109.128/ | tee reports/direct_page_test.html
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100  207 100    207  0      0   4547    0      0      0      0     0
<!DOCTYPE html>
<html><body style="font-family:Arial"><h1>ICDFA DNS Spoofing Training Page</h1><p>This is an authorized simulation. Do not enter credentials.</p><p>Analyst: Kumpon Emmanuel</p></body></html>

(ponzhi@kali)-[~/SBT-DF203-Lab8]
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
sed: can't read scripts/arp.py: No such file or directory

(ponzhi@kali)-[~/SBT-DF203-Lab8]
$
```

Script Review arp.py

Before running the script, please be certain to review the content. The script arp.py has been fully read. It carries out ARP cache poisoning using Scapy. The function get_mac() finds a host's MAC address by sending a broadcast ARP request. The function spoof() sends fake ARP replies, making the target think that the analyst VM controls the gateway's IP address. This allows the analyst to intercept communication between the target and the gateway.

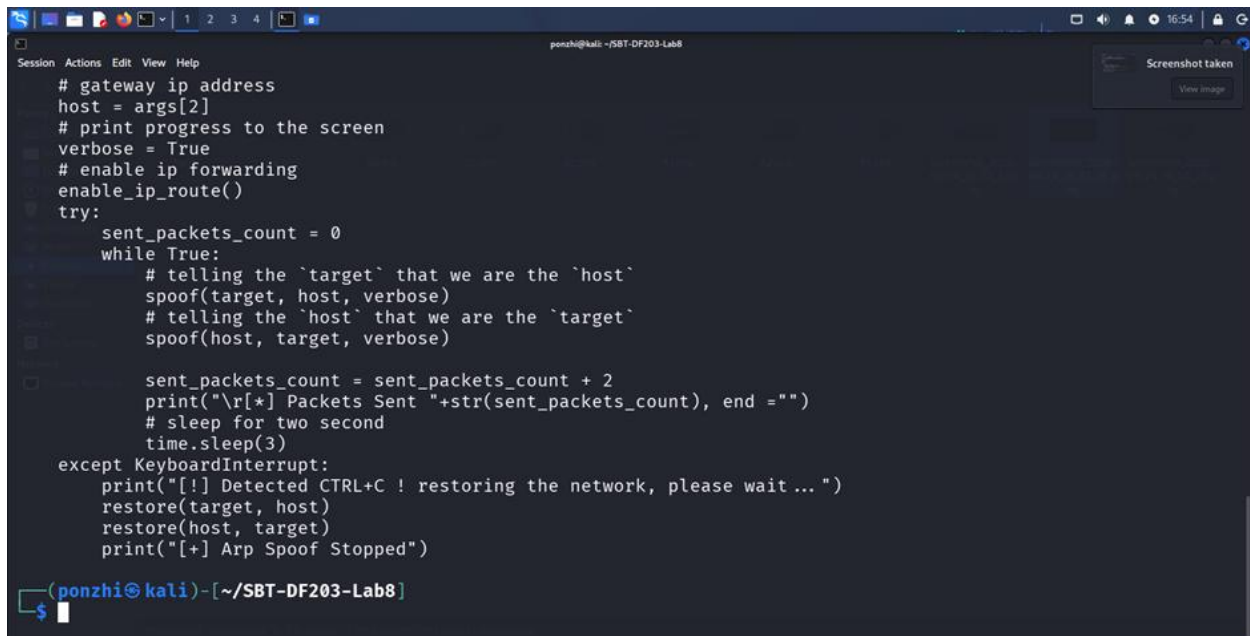


```
(ponzhi@kali)-[~/SBT-DF203-Lab8]
$ sed -n '1,240p' scripts/arp.py | tee reports/arp_script_review.txt
# https://www.geeksforgeeks.org/python-how-to-create-an-arp-spoofers-using-scapy/
# https://www.thepythoncode.com/article/building-arp-spoofers-using-scapy

import scapy.all as scapy
import time
import argparse
import time
import os
import sys

def get_mac(ip):
    arp_request = scapy.ARP(pdst = ip)
    broadcast = scapy.Ether(dst = "ff:ff:ff:ff:ff:ff")
    arp_request_broadcast = broadcast / arp_request
    answered_list = scapy.srp(arp_request_broadcast, timeout = 5, verbose = False)[0]
    return answered_list[0][1].hwsrc

def spoof(target_ip, host_ip, verbose=True):
    """
    Kali tells `target_ip` saying that "I have the host_ip that you want to talk".
    However, Kali provides its own Mac address.
    In target's arp:
    Host IP, attacker's/Kali Mac address
    """
```



```
# gateway ip address
host = args[2]
# print progress to the screen
verbose = True
# enable ip forwarding
enable_ip_route()
try:
    sent_packets_count = 0
    while True:
        # telling the `target` that we are the `host`
        spoof(target, host, verbose)
        # telling the `host` that we are the `target`
        spoof(host, target, verbose)

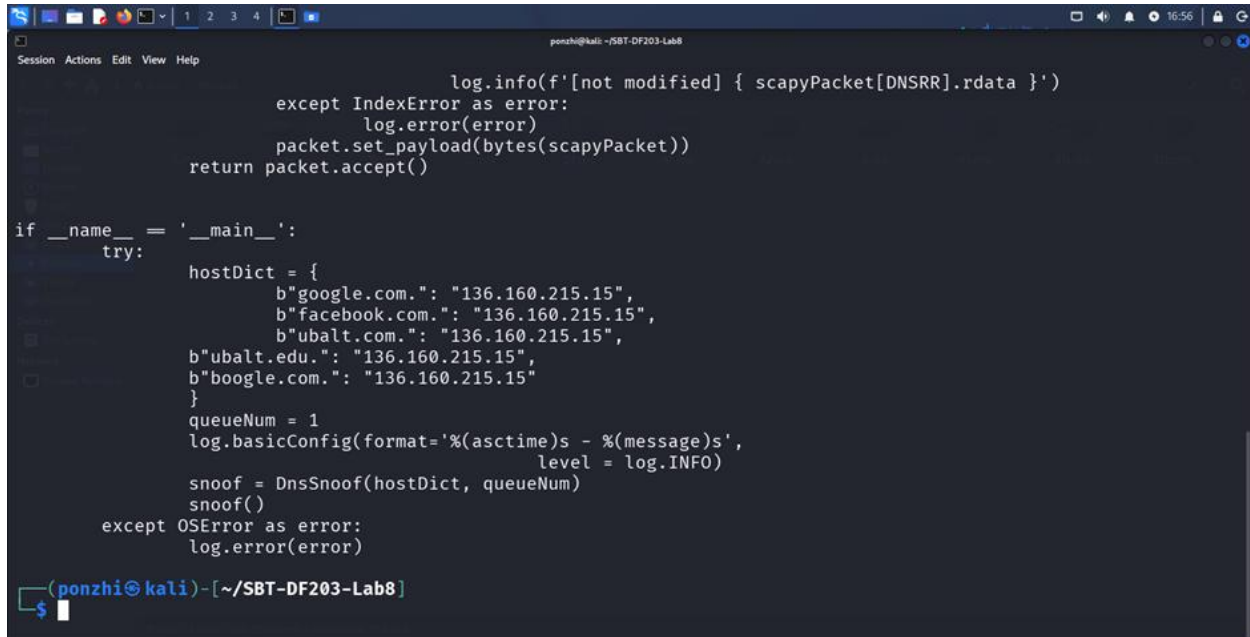
        sent_packets_count = sent_packets_count + 2
        print("\r[*] Packets Sent "+str(sent_packets_count), end = "")
        # sleep for two second
        time.sleep(2)
except KeyboardInterrupt:
    print("[!] Detected CTRL+C ! restoring the network, please wait...")
    restore(target, host)
    restore(host, target)
    print("[+] Arp Spoof Stopped")

(ponzhi@kali)-[~/SBT-DF203-Lab8]
$
```

No hardcoded targets were found in arp.py; target and gateway IPs are supplied as command-line arguments at runtime, consistent with the intended usage.

Script Review dns_spoof.py

The hostDict in dns_spoof.py was set up with real, public domains instead of the reserved training domain. This goes against the safety rule in the lab manual, which states, "Never use a real bank or public domain."



```
log.info(f'[not modified] { scapyPacket[DNSRR].rdata }')
except IndexError as error:
    log.error(error)
    packet.set_payload(bytes(scapyPacket))
return packet.accept()

if __name__ == '__main__':
    try:
        hostDict = {
            b"google.com.": "136.160.215.15",
            b"facebook.com.": "136.160.215.15",
            b"ubalt.com.": "136.160.215.15",
            b"ubalt.edu.": "136.160.215.15",
            b"boogle.com.": "136.160.215.15"
        }
        queueNum = 1
        log.basicConfig(format='%(asctime)s - %(message)s',
                        level = log.INFO)
        snoof = DnsSnoof(hostDict, queueNum)
        snoof()
    except OSError as error:
        log.error(error)
```

(ponzhi@kali) - [~/SBT-DF203-Lab8]
\$

This was corrected before any execution was attempted. hostDict was edited to contain only the reserved training domain, mapped to the analyst VM's own IP:

```
hostDict = { b" portal.icdfa.test.": "192.168.109.128" }
```

The corrected dns_spoof.py was re-reviewed and logged to reports/dns_script_review.txt. On the first execution attempt, it failed with ModuleNotFoundError: No module named 'netfilterqueue'; libnetfilter-queue-dev was subsequently installed via apt (see Figure 3) to address the underlying dependency, though the Python module import has not yet been re-verified.